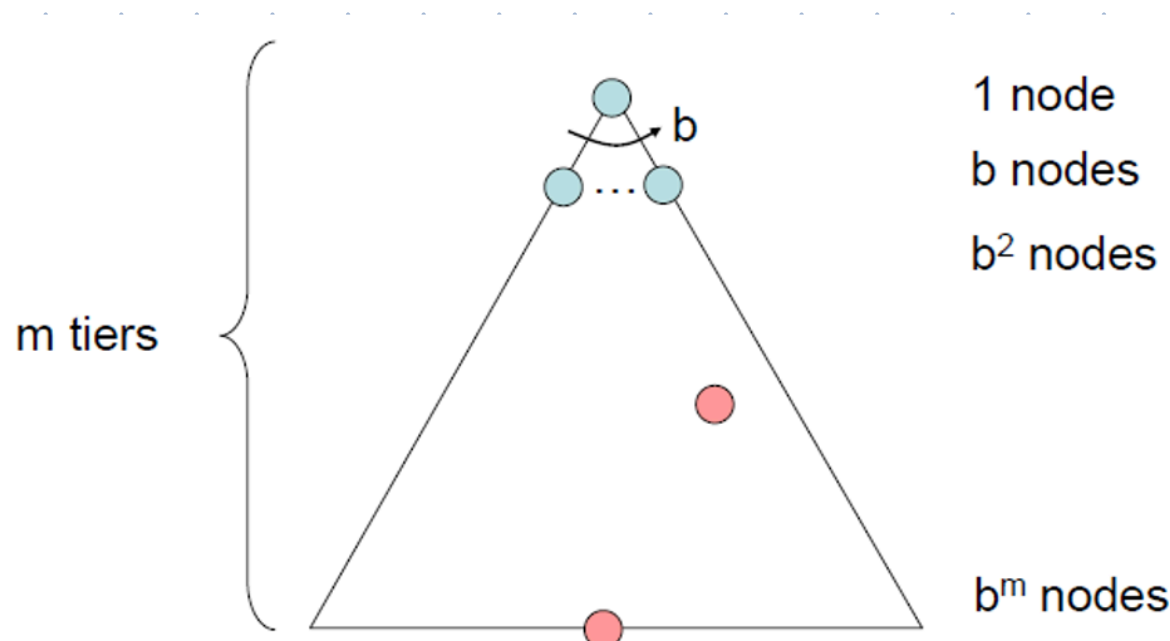


Uninformed Search:

- ↳ Also called blind search
- ↳ They have no additional about states beyond that provided in the problem definition.
- ↳ All they can do is generate successors & distinguish goal state from a non-goal state.

Search strategies:

- ↳ A search strategy is defined by picking the order of state expansion.
- ↳ Strategies are evaluated along four dimensions
 1. Completeness: Does it always find a solution if it exists?
 2. Time complexity: No. of nodes expanded during execution
 3. Space complexity: Maximum no. of nodes or memory required to store the expanded nodes
 4. Optimal: Does it always find the least cost solution?
- ↳ Time & space complexity are measured in terms of:
 - b : Max branching factor
 - d : Depth of the least cost solution
 - m : Max depth of the state space (maybe ∞)



No. of nodes in entire tree?

$$1 + b + b^2 + \dots + b^m$$

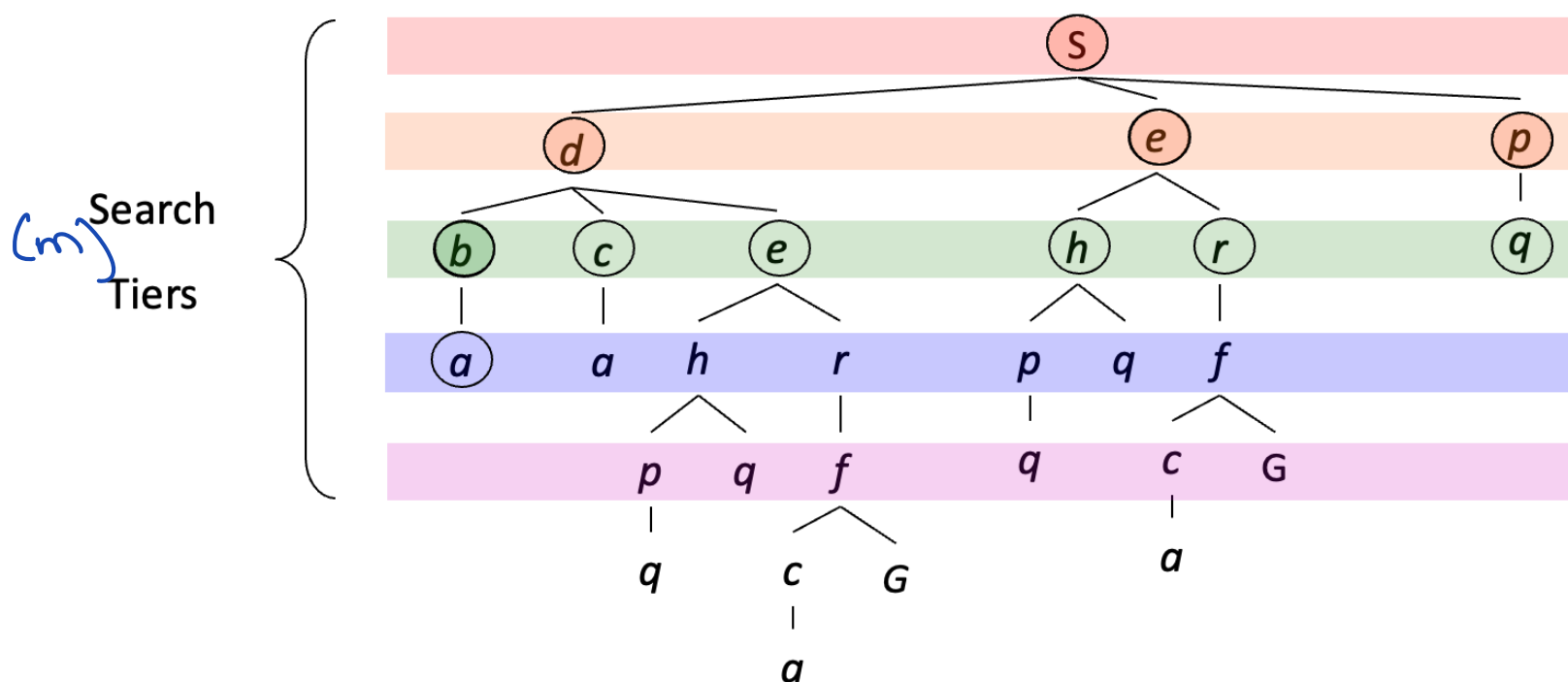
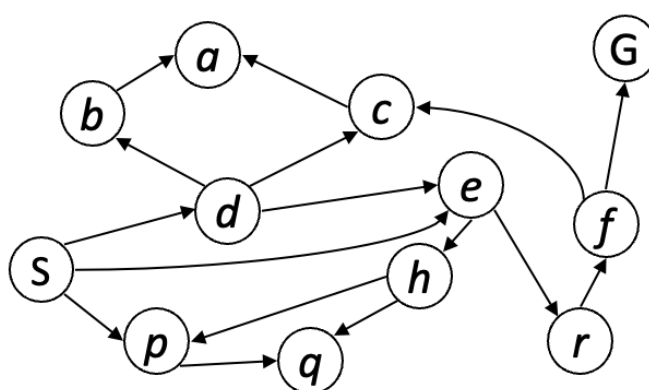
$$O(b^m)$$

Breadth-first search

Strategy: expand a shallowest node first

Implementation:

Frontier is a FIFO queue



Time complexity: $O(b^s)$

Space complexity: $O(b^s)$

where s is the depth of the shallowest solution.

Completeness: s must be finite if a solution exists, yes!

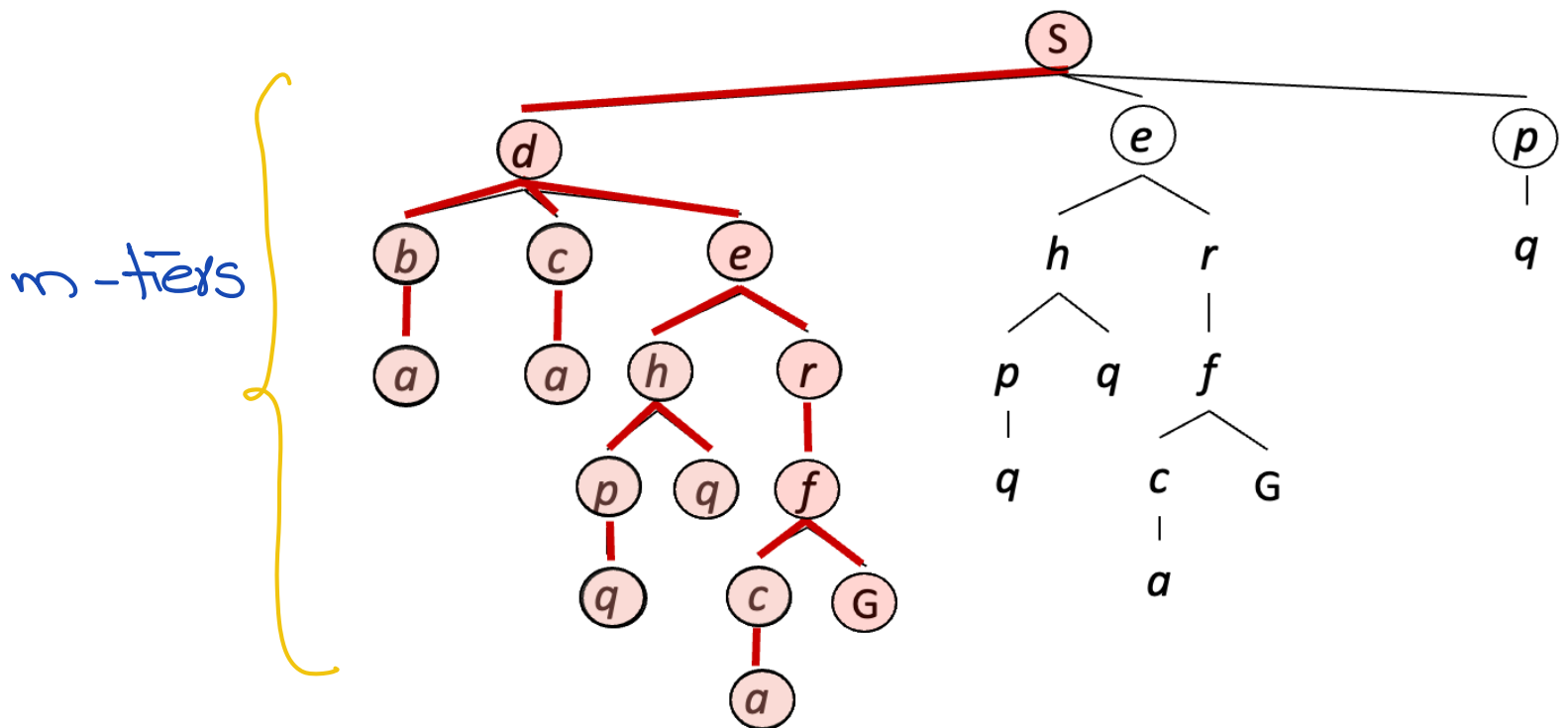
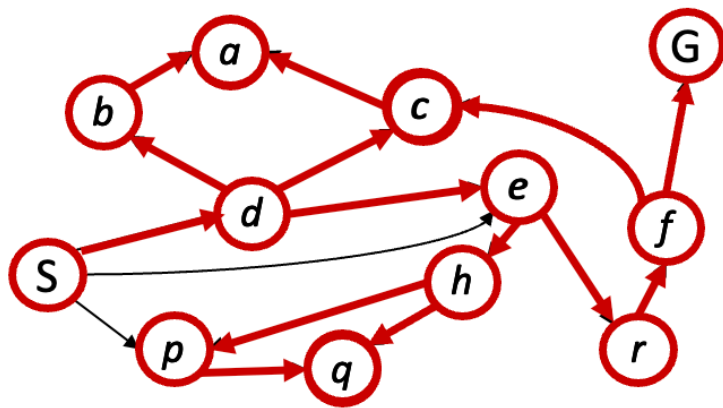
Optimal: only if all path costs are same also No!

Depth-First Search

Strategy: expand a deepest node first

Implementation:

Frontier is a LIFO stack



Time complexity: $O(b^m)$

Space complexity: $O(b \times m)$

Completeness: If search tree is infinite, DFS can get stuck.

Optimal: only finds the left-most solution, No!

Depth-limited search

↳ DFS fails if search tree is infinite, so we limit search depth to l .

↳ That is, nodes at depth l are treated as if they have no children.

Iterative Deepening Search

↳ Run Depth-limited search with increasing depth-limits.

$l=1$

while no solution, do

DFS from initial state so with cutoff L If found goal,

stop & return solution

else, increment depth limit L

Time complexity: $O(b^L)$

Space complexity: $O(b \times L)$

Completeness: Yes!

Optimal: only if all path costs are same else No!

Uniform cost search:

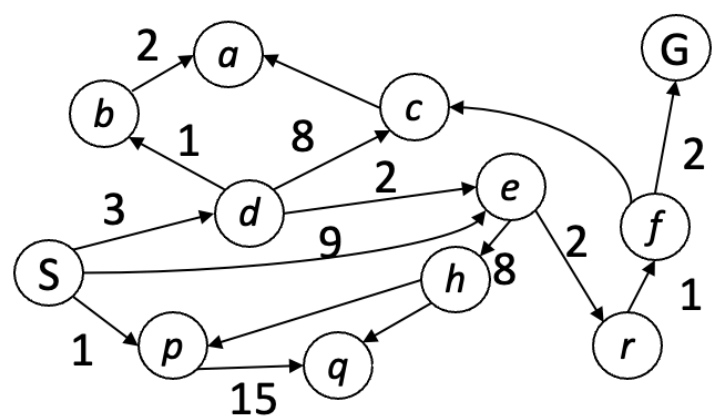
Let $g(n)$ = cost from start state S to node n

Always expand the node on the fringe with minimum cost $g(n)$

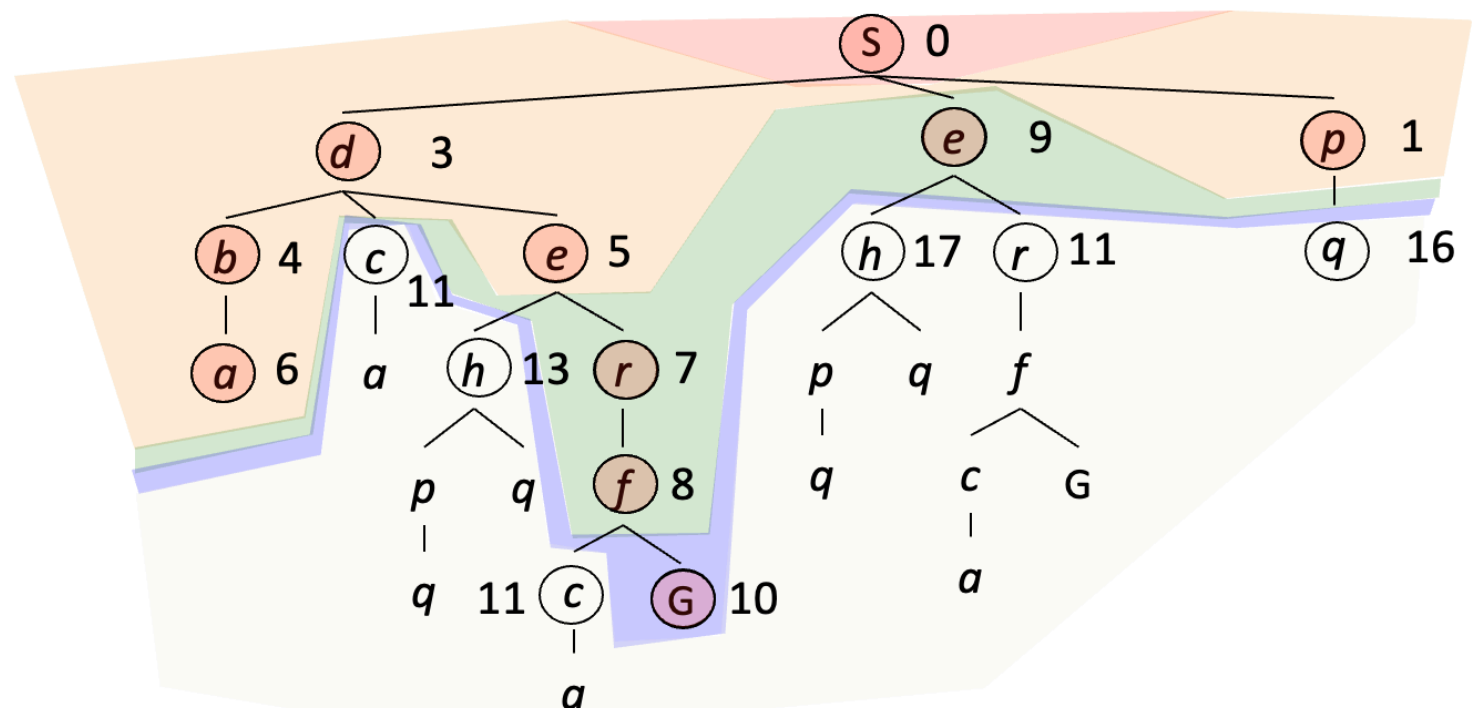
Note that if costs are equal, then it is BFS

Strategy: expand a cheapest node first:

Frontier is a priority queue
(priority: cumulative cost)



Cost contours



Time complexity: $O(b^{C^*/\epsilon})$

Space complexity: $O(b^{C^*/\epsilon})$

Completeness: yes

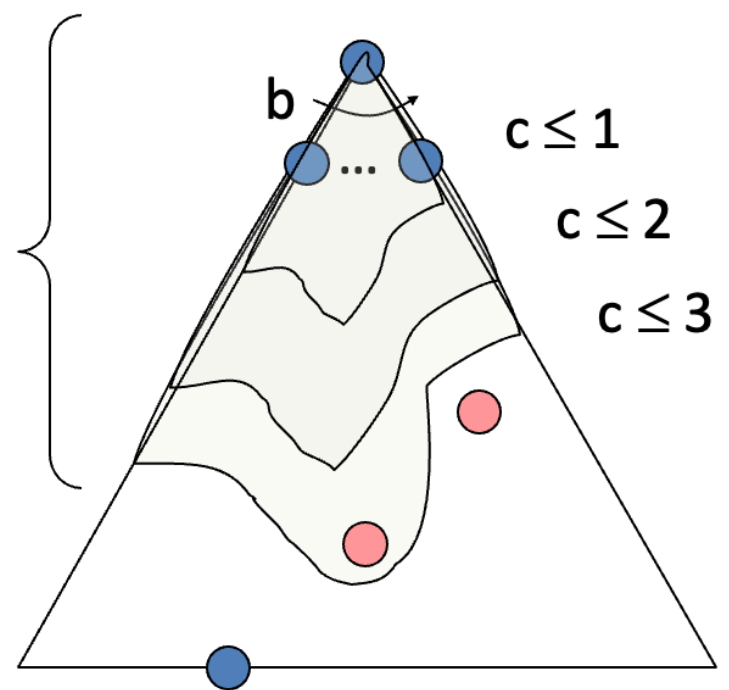
Optimal: yes

$$d + \epsilon \leq C^*$$

$$d \leq \frac{C^*}{\epsilon}$$

may find solution before ($<$) but at max it will go $\frac{C^*}{\epsilon}$

C^*/ϵ "tiers"



Search	Frontier	Completeness	Optimality	Time	Space
DFS (Depth-First)	Stack	tree search - no (cycle) graph search - yes (finite) no (infinite)	no	$O(b^m)$	$O(bm)$
BFS (Breadth-First)	Queue	yes	no (except when all edge costs same)	$O(b^s)$	$O(b^s)$
Iterative Deepening (DFS result w/ modified DFS algo)	Stack (same as DFS)	yes (same as BFS)	no (same as BFS)	$O(b^s)$ (same as BFS)	$O(bs)$ (same as DFS but w/ shortest solution length)
UCS (Uniform Cost)	heap-based PQ (backward cost)	yes (assuming positive edge costs and $\epsilon > 0$)	yes (assuming positive edge costs and $\epsilon > 0$)	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$

b = branching factor
(assume finite)

m = max depth of
search tree

s = smallest depth
of solution
(assume finite)

C^* = cost of
optimal solution
(assume finite)

ϵ = minimum cost
between 2 nodes

